



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
CIENCIAS DE LA INGENIERIA
INGENIERIA DE SOFTWARE
Semestre 2026-2

Documentación

Profesor: Dr. Oscar Francisco Fuentes
Casarrubias

Equipo :

Integrantes:

- Carpizo Garduño Andrés
- Estanislao Martínez Johan
- León Zárate Rubí Margarita

Grupo: 5

Fecha de entrega:



Objetivo:

Reafirmar lo aprendido en la teoría y práctica sobre los sistemas de numeración posicional mediante el desarrollo de un programa en lenguaje python. Se busca aplicar los algoritmos clásicos de conversión de bases por divisiones y multiplicaciones sucesivas, así como simular el procesamiento aritmético con signo en un sumador binario de 6 bits utilizando la regla del complemento a 2 y la evaluación de acarreos.

Introducción:

En la electrónica digital y el diseño de sistemas computacionales, comprender el manejo de la información a nivel interno es indispensable. Aunque como usuarios interactuamos habitualmente con el sistema numérico decimal, las computadoras procesan, almacenan y realizan operaciones utilizando exclusivamente lógica binaria. Por tal razón, el estudio de las conversaciones entre distintas bases numéricas (como octal, hexadecimal y binario) y la representación de números con signo es un pilar fundamental de esta asignatura.

En esta práctica se aborda la implementación en software de dos módulos principales. El primero es un convertidor de bases generales que procesa tanto la parte entera como la fraccionaria mediante el método de divisiones y multiplicaciones sucesivas, incluyendo un control para validar que las entradas sean acordes a la base seleccionada. El segundo módulo es un emulador de un sumador binario de 6 bits que utiliza la representación en Complemento a 2 para manejar números negativos, simulando a nivel lógico el comportamiento de los circuitos aritméticos mediante la propagación de acarreos bit a bit y la detección de desbordamiento (overflow).

Desarrollo Metodológico:

Problema 1: Conversión de Bases y Validación de Datos

Validación de la Entrada: Antes de realizar cualquier cálculo, el programa analiza la cadena ingresada. La función recorre cada carácter y verifica que su valor no sea igual o mayor a la base de origen especificada. Si se detecta un carácter no permitido (por ejemplo, un dígito 8 cuando la base elegida es octal), el programa detiene el proceso y lanza una alerta. Además, se implementó el mapeo estándar de letras de la A a la F para dar soporte a bases superiores a 10 (hasta base 16).

```

# VALIDACION DE QUE EL NUMERO SEA VALIDO EN LA BASE QUE EL USUARIO DIO

def validar_numero(numero_str, base): 1 usage
    # quitamos el signo negativo y el punto decimal para revisar solo digitos
    texto = numero_str
    if texto.startswith('-'):
        texto = texto[1:]
    texto = texto.replace('.', '')

    # recorremos caracter por caracter
    for i in range(len(texto)):
        digito = texto[i]
        valor = caracter_a_valor(digito)
        if valor >= base:
            # si el digito no cabe en la base, mandamos error
            raise ValueError("Error: el digito '" + digito + "' no es valido en base " + str(base))
    return True

```

```

# FUNCIONES DE APOYO PARA CONVERTIR CARACTERES A NUMEROS Y VICEVERSA

def caracter_a_valor(c): 3 usages
    # si el caracter es un numero del 0 al 9, regresamos su valor directo
    if c >= '0' and c <= '9':
        return int(c)
    # si no, es una letra (A-F) y la convertimos a su valor (A=10, B=11, etc)
    c = c.upper()
    return ord(c) - ord('A') + 10

def valor_a_caracter(v): 2 usages
    # si el valor es menor a 10 lo regresamos como numero normal
    if v < 10:
        return str(v)
    # si es mayor o igual a 10 lo pasamos a letra
    return chr(v - 10 + ord('A'))

```

Conversión a Decimal (“La Cuenta Larga”): Se optó por no utilizar funciones integradas del lenguaje para las conversiones de base. En su lugar, el número se divide en su parte entera y fraccionaria. La parte entera se procesa evaluando los dígitos de derecha a izquierda multiplicados por la base de origen elevada a su posición i ($base^i$). La parte fraccionaria se procesa de izquierda a derecha multiplicando cada dígito por la base elevada a un exponente negativo ($base^{-(i+1)}$).

```

47 # ALGORITMO DE LA CUENTA LARGA
48
49 def cuenta_larga_a_decimal(numero_str, base_origen): 3 usages
50     # primero validamos que todos los digitos sean correctos para esa base
51     validar_numero(numero_str, base_origen)
52
53     negativo = False
54     if numero_str.startswith('-'):
55         negativo = True
56         numero_str = numero_str[1:]
57
58     # separamos parte entera y parte fraccionaria si tiene punto
59     if '.' in numero_str:
60         partes = numero_str.split('.')
61         parte_entera = partes[0]
62         parte_frac = partes[1]
63     else:
64         parte_entera = numero_str
65         parte_frac = ""
66
67     valor_total = 0.0
68
69     # recorremos la parte entera de derecha a izquierda
70     n = len(parte_entera)
71     for i in range(n):
72         digito = parte_entera[n - 1 - i] # tomamos desde el ultimo caracter
73         valor_total = valor_total + caracter_a_valor(digito) * (base_origen ** i)
74
75     # recorremos la parte fraccionaria de izquierda a derecha
76     for i in range(len(parte_frac)):
77         digito = parte_frac[i]
78         valor_total = valor_total + caracter_a_valor(digito) * (base_origen ** -(i + 1))
79
80     if negativo:
81         return -valor_total
82     else:
83         return valor_total

```

Conversión de Decimal a Base Destino: Con el valor transformado a decimal, se aplica el método clásico: la parte entera se convierte mediante divisiones sucesivas entre la base de destino guardando los residuos, mientras que la parte fraccionaria se multiplica sucesivamente por la base destino reteniendo las partes enteras resultantes como los dígitos del nuevo número.

```

86 # CONVERTIR UN VALOR DECIMAL A LA BASE DESTINO
87
88 def decimal_a_base(valor, base_destino, precision=6): 1 usage
89     if valor == 0:
90         return "0"
91
92     negativo = False
93     if valor < 0:
94         negativo = True
95         valor = abs(valor)
96
97     parte_entera = int(valor)
98     parte_frac = valor - parte_entera
99
100     # divisiones sucesivas para la parte entera
101     resultado_entero = ""
102     if parte_entera == 0:
103         resultado_entero = "0"
104
105     while parte_entera > 0:
106         residuo = parte_entera % base_destino
107         resultado_entero = valor_a_caracter(residuo) + resultado_entero
108         parte_entera = parte_entera // base_destino
109
110     # si no hay parte fraccionaria, ya terminamos
111     if parte_frac == 0:
112         if negativo:
113             return "-" + resultado_entero
114         else:
115             return resultado_entero

```

```

116
117     # multiplicaciones sucesivas para la parte fraccionaria
118     resultado_frac = ""
119     contador = 0
120     while parte_frac > 0 and contador < precision:
121         parte_frac = parte_frac * base_destino
122         digito = int(parte_frac)
123         resultado_frac = resultado_frac + valor_a_caracter(digito)
124         parte_frac = parte_frac - digito
125         contador = contador + 1
126
127     resultado_final = resultado_entero + "." + resultado_frac
128     if negativo:
129         return "-" + resultado_final
130     else:
131         return resultado_final

```

Problema 2: Emulador de Sumador Binario de 6 bits

El desarrollo de la simulación de la suma binaria se enfocó en emular el funcionamiento a nivel de hardware mediante un diseño modular:

Función de Suma de 1 Bit (sumar_un_bit): Se definió un bloque básico que recibe dos bits operandos y un acarreo de entrada. Mediante operaciones aritméticas simples, calcula la suma total y obtiene el bit resultante utilizando el operador residuo (%2). Si la suma acumulada es mayor o igual a 2, se genera un acarreo de salida (1), de lo contrario se mantiene en 0.

```
def sumar_un_bit(bit_a, bit_b, acarreo_in): 3 usages
    # convertimos a entero
    suma_total = int(bit_a) + int(bit_b) + acarreo_in # esto da un numero entre 0 y 3

    suma = suma_total % 2 # el resultado del bit es el residuo de dividir entre 2

    if suma_total >= 2:
        acarreo_out = 1
    else:
        acarreo_out = 0

    return str(suma), acarreo_out

def decimal_a_binario_magnitud(valor, bits=5): 1 usage
    # convierte solo la magnitud a binario de bits
    v = abs(valor)
    binario = ""
    for i in range(bits):
        binario = str(v % 2) + binario
        v = v // 2
    return binario
```

Representación en Complemento a 2: El programa acepta números en el rango de -32 a 31. Para los valores positivos, simplemente se calcula su representación binaria de 5 bits de magnitud y se les antepone un bit de signo 0. Cuando el número es negativo, se obtiene la magnitud en binario, se invierten todos sus bits (cambiando 0 por 1 y viceversa) y se le suma un 1 al bit menos significativo (LSB) apoyándose en la función de 1 bit. Finalmente, se antepone el bit de signo 1.

```

160 def complemento_a_dos(valor, bits_totales=6): 2 usages
161     bits_magnitud = bits_totales - 1
162
163     # checamos que el valor quepa en el rango permitido
164     limite_inferior = -(2 ** bits_magnitud)
165     limite_superior = (2 ** bits_magnitud) - 1
166     if valor < limite_inferior or valor > limite_superior:
167         raise OverflowError("El valor " + str(valor) + " no cabe en " + str(bits_totales) + " bits.")
168
169     magnitud = decimal_a_binario_magnitud(valor, bits_magnitud)
170
171     # si es positivo o cero, el bit de signo es 0 y ya no hacemos nada mas
172     if valor >= 0:
173         return "0" + magnitud
174
175     # si es negativo hacemos el complemento a 2
176
177     # paso 1: invertimos cada bit
178     magnitud_invertida = ""
179     for i in range(len(magnitud)):
180         if magnitud[i] == '0':
181             magnitud_invertida = magnitud_invertida + '1'
182         else:
183             magnitud_invertida = magnitud_invertida + '0'
184
185     # paso 2: le sumamos 1 al bit menos significativo usando nuestra funcion de 1 bit
186     magnitud_c2 = ""
187     acarreo = 1 # empezamos con acarreo 1 porque estamos sumando un 1
188     for i in range(len(magnitud_invertida) - 1, -1, -1):
189         bit_actual = magnitud_invertida[i]
190         s, acarreo = sumar_un_bit(bit_actual, bit_b: '0', acarreo)
191         magnitud_c2 = s + magnitud_c2
192
193     return "1" + magnitud_c2 # el bit de signo es 1 porque es negativo
194

```

Bucle de Suma y Detección de Overflow: La suma de ambos operandos se realiza en 6 etapas secuenciales. Primero, mediante un ciclo de 5 iteraciones, se procesan los bits de magnitud propagando el acarreo de una columna a la siguiente. Posteriormente, se efectúa una 6.^a llamada a la función para sumar los bits de signo (MSB) junto con el último acarreo generado. El desbordamiento (overflow) se detecta comparando el acarreo que ingresó al bit de signo con el acarreo que salió de él; si ambos acarreos son distintos, se notifica al usuario que ocurrió un overflow aritmético.


```

195
196 def sumador_6_bits(num1_str, num2_str, base): 1 usage
197     # primero pasamos ambos numeros a decimal usando la funcion del problema 1
198     valor1 = int(cuenta_larga_a_decimal(num1_str, base))
199     valor2 = int(cuenta_larga_a_decimal(num2_str, base))
200
201     binario1 = complemento_a_dos(valor1)
202     binario2 = complemento_a_dos(valor2)
203
204     resultado = ""
205     acarreo = 0
206
207     # las posiciones 1 a 5 de la cadena son la magnitud
208     for i in range(5, 0, -1):
209         suma, acarreo = sumar_un_bit(binario1[i], binario2[i], acarreo)
210         resultado = suma + resultado
211
212     # guardamos el acarreo que entra a la columna de signo, lo necesitamos para el overflow
213     acarreo_que_entra_al_signo = acarreo
214
215     # sumamos los bits de signo de los dos numeros
216     suma_signo, acarreo_que_sale_del_signo = sumar_un_bit(binario1[0], binario2[0], acarreo)
217     resultado = suma_signo + resultado
218
219     # si el acarreo que entro es distinto al que salio, hubo overflow
220     if acarreo_que_entra_al_signo != acarreo_que_sale_del_signo:
221         hubo_overflow = True
222     else:
223         hubo_overflow = False
224
225     return binario1, binario2, resultado, hubo_overflow
226
227

```

Capturas del funcionamiento del código:

Problema 1: Conversión de Bases

Algoritmo matemático de conversión posicional e implementación de divisiones/multiplicaciones sucesivas para fracciones


```

--- MENU PRINCIPAL ---
1. Conversion de bases (divisiones y multiplicaciones sucesivas)
2. Sumador binario de 6 bits (complemento a 2)
3. Salir
Elige una opcion: 1

-- Conversion de base --
Base de origen (2-16): 16
Numero a convertir: A.8
Base de destino (2-16): 2
Resultado: 1010.1

```

Problema 1: Validación de Datos

Detección y rechazo de caracteres inválidos según la base de origen elegida (ej. rechazar '9' en base 9).

```

--- MENU PRINCIPAL ---
1. Conversion de bases (divisiones y multiplicaciones sucesivas)
2. Sumador binario de 6 bits (complemento a 2)
3. Salir
Elige una opcion: 1

-- Conversion de base --
Base de origen (2-16): 8
Numero a convertir: 1789
Base de destino (2-16): 10
Error: Error: el dígito '8' no es valido en base 8

```

```

--- MENU PRINCIPAL ---
1. Conversion de bases (divisiones y multiplicaciones sucesivas)
2. Sumador binario de 6 bits (complemento a 2)
3. Salir
Elige una opcion: 1

-- Conversion de base --
Base de origen (2-16): 9
Numero a convertir: 179
Base de destino (2-16): 10
Error: Error: el dígito '9' no es valido en base 9

```

Problema 2: Función de 1-bit

Función modular de un bit que use exclusivamente operaciones aritméticas básicas y condicionales sencillos.

```
--- MENU PRINCIPAL ---
1. Conversion de bases (divisiones y multiplicaciones sucesivas)
2. Sumador binario de 6 bits (complemento a 2)
3. Salir
Elige una opcion: 2

-- Sumador de 6 bits (C2) --
Base en la que estan los numeros: 10
Primer numero (rango -32 a 31): 1
Segundo numero (rango -32 a 31): 1

Operando 1 (6 bits): 000001
Operando 2 (6 bits): 000001
Suma binaria:      000010
Operacion correcta, sin overflow.
```

Problema 2: Complemento a 2

Inversión de bits y adición de +1 en LSB

```
--- MENU PRINCIPAL ---
1. Conversion de bases (divisiones y multiplicaciones sucesivas)
2. Sumador binario de 6 bits (complemento a 2)
3. Salir
Elige una opcion: 2

-- Sumador de 6 bits (C2) --
Base en la que estan los numeros: 10
Primer numero (rango -32 a 31): 12
Segundo numero (rango -32 a 31): -5

Operando 1 (6 bits): 001100
Operando 2 (6 bits): 111011
Suma binaria:      000111
Operacion correcta, sin overflow.
```

Problema 2: Bucle de Suma de 6 bits

Detección de números negativos y conversión automática mediante inversión de bits y adición de +1 en el LSB.

```
--- MENU PRINCIPAL ---
1. Conversion de bases (divisiones y multiplicaciones sucesivas)
2. Sumador binario de 6 bits (complemento a 2)
3. Salir
Elige una opcion: 2

-- Sumador de 6 bits (C2) --
Base en la que estan los numeros: 10
Primer numero (rango -32 a 31): 20
Segundo numero (rango -32 a 31): 15

Operando 1 (6 bits): 010100
Operando 2 (6 bits): 001111
Suma binaria:      100011
ADVERTENCIA: hubo overflow (desbordamiento aritmetico).
```

Conclusión:

El desarrollo de esta práctica permitió consolidar la relación directa entre la teoría de los sistemas de numeración y su implementación práctica mediante lógica de programación. Se logró comprobar que el método de la “cuenta larga”, junto con el esquema de divisiones y multiplicaciones sucesivas, constituye un algoritmo robusto y universal para la conversión entre bases arbitrarias, permitiendo el tratamiento adecuado tanto de magnitudes enteras como de partes fraccionarias. Por otro lado, la emulación del sumador binario de 6 bits utilizando hardware se reducen esencialmente a estructuras iterativas simples respaldadas por bloques de suma de un solo bit. La correcta propagación de acarreo y la evaluación comparativa entre el acarreo de entrada y salida del bit de signo demostraron ser un mecanismo preciso para la detección de desbordamientos aritméticos (overflow).

La realización de esta tarea resultó ser un ejercicio de gran valor académico. En muchas ocasiones utilizamos funciones integradas en los lenguajes de programación o en calculadoras sin profundizar en el mecanismo interno que las hace funcionar. Implementar desde cero la validación de caracteres por base y construir el algoritmo de conversión mediante aritmética de posiciones me permitió entender con claridad los errores de precisión que pueden surgir al trabajar con partes fraccionarias en distintas bases.